

Detecção de Deadlocks e Violação da Ordem de Aquisição de Travas

Baseado em DEFINITION.md

Universidade Federal do Rio Grande do Norte

16 de julho de 2025

Integrantes

Nome	E-mail
Corpo docente	
Samuel Xavier de Souza	samuel.xavier@ufrn.br
Wedson Torres de Almeida Filho	wedson.almeida@dca.ufrn.br
Corpo discente	
Vinicius de Lima Duarte Sampaio	vinicius.lima.115@ufrn.edu.br
Emanuel Rawa Gurgel de Araujo Freitas	emanuelrawa@gmail.com
Arthur Jose Arruda Skeete	arthurjoskeete@gmail.com
Yuri Maximiliano Brasileiro Santos	yurimax322@gmail.com
José Henrique Targino Dias Góis	ze.gois.00@gmail.com

Sumário

- 1 Introdução e Objetivo
- 2 Fundamentos Teóricos
- 3 Implementação por Arquivo
- 4 Exemplo de Execução
- 5 Conclusão e Extensões

Objetivo Geral

- Estender mutexes didáticos para detectar deadlocks e violações de ordem.
- Inspirado no lockdep do Linux.
- Abordagens: Grafo de espera (reativo) ou verificação de ordem (preventivo).

Requisitos Principais

- Base em mutex C.
- Estruturas: Dono atual, fila de espera, ordem por thread.
- Implementar pelo menos uma abordagem (extra para ambas).
- Foco aqui: Lockdep-style (preventivo); grafo de espera como extensão.

Conceitos Chave

- Deadlock: Espera circular (condições de Coffman).
- Grafo de Espera: Nodes=threads/resources; ciclo=deadlock real (DFS).
- Lockdep-Style: Grafo dependências (nodes=locks, edges=ordem); ciclo=potencial inversão.

lockdep.h: Estruturas de Dados

- lock_{node_t} : *Grafo(children) + lista(next); dono(current_heldlock).*
- adjacency_locks_t : *Edges para dependências.*
- held_lock_t : *Pilha de holds com trace timestamp.*
- thread_context_t : *Contexto de thread com pilha held.*

```
#include <time.h>

#define LOCKDEP_MAX_STACK_DEPTH 8
#define LOCKDEP_MAX_LOCK_NAME 64

typedef struct adjacent_locks adjacency_locks_t;
```

pthread; *nterpose.c* : *Hooking Pthread*

- Interposição via dlsym para lock/unlock/trylock.
- Chama lockdep_acquire/release; evitarecurso(in;nterpose).
- Para trylock: Prevene se deadlock detectado.

```

    init_real_functions();
}

/// The lockdep uses a mutex to protect its internal state, so we use this to
/// avoid recursing lockdep validation across itself.
static __thread bool in_interpose = false;

int pthread_mutex_lock(pthread_mutex_t* mutex)
{
    init_real_functions();

```


lockdep_core.c : *Lógica Central*

- `acquire_lock` : *Registra, adiciona dep, checa ciclo (DFS).*
- `has_cycle` : *DFS recursivo para detectar ciclos; reporta com traces.*
- `print_cycle` : *Relatório detalhado (path, owners, sugestão de ordem).*

```

"=====\n"
"[LOCKDEP] *** DEADLOCK DETECTED ***\n"
"=====\n");

fprintf(stderr, "\n[LOCKDEP] Deadlock cycle detected:\n");
for (size_t i = 0; i < cycle_path_size; i++) {
    lock_node_t* current_lock = cycle_path[i];
    fprintf(stderr, "  [%zu] %s\n", i + 1, current_lock->lock_name);

    if (current_lock->current_held_lock) {
        held_lock_t* held = current_lock->current_held_lock;

```

t05_dining_pphilosophers.c : Exemplo

- Clássico problema: 5 filósofos, 5 garfos (mutexes).
- $SHALL_LOCK = 1$: Todos pegam left first β ciclo.
- Com lockdep: Detecta preventivamente.

```

if (id == NUM_PHILOSOPHERS - 1) {
    // Last philosopher picks up right fork first
    first_fork = right_fork;
    second_fork = left_fork;
} else {
    // Other philosophers pick up left fork first
    first_fork = left_fork;
    second_fork = right_fork;
}
}

```

Resultado no Exemplo

- Com $LD_P RELOAD$: *Detecta ciclo* (e.g., $lock_0 \times 8050 \beta \dots \beta lock_0 \times 8050$).
- Reporta: Path, tempos, traces, states, sugestão (order by addr).
- Atende preventivo; extensão para reativo via waiters.

Conclusão

- Atende lockdep-style; demonstra grafos em concurrency.
- Limitações: $O(n)$ lookups (add hash); sem fila wait full.
- Impacto: Educacional; usa IA para análise.

Extensões Futuras

- Implementar grafo espera: Add fila waiters, DFS thread-lock.
- Suporte a outros sync (condvars, RW locks).
- Otimizações: Hash para lookups; integração gdb/valgrind.

Perguntas?

Obrigado! Dúvidas?